Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3773274.3774276

RESEARCH-ARTICLE

Energy-Aware Latency Optimization for Scheduling Serverless Workload in Edge Computing Environment

SHRIJAN SUBEDI, Leeds Beckett University, Leeds, West Yorkshire, U.K.

SATISH KUMAR, Leeds Beckett University, Leeds, West Yorkshire, U.K.

NAWAR JAWAD, Leeds Beckett University, Leeds, West Yorkshire, U.K.

AH LIAN KOR, Leeds Beckett University, Leeds, West Yorkshire, U.K.

Open Access Support provided by:

Leeds Beckett University

PDF Download
3773274.3774276.pdf
02 March 2026
Total Citations: 0
Total Downloads: 270

Published: 31 December 2025

Citation in BibTeX format

UCC '25: 2025 IEEE/ACM 18th International Conference on Utility and Cloud Computing
December 1 - 4, 2025
France, France

Conference Sponsors:
SIGARCH

Energy-Aware Latency Optimization for Scheduling Serverless Workload in Edge Computing Environment

Shrijan Subedi
Leeds Beckett University
Leeds, Nepal
shrnsubedi22@gmail.com

Satish Kumar
Leeds Beckett University, UK
Leeds, United Kingdom
s.kumar@leedsbeckett.ac.uk

Nawar Jawad
Leeds Beckett University, UK
Leeds, United Kingdom
n.jawad@leedsbeckett.ac.uk

Ah Lian Kor
Leeds Beckett University, UK
Leeds, United Kingdom
A.Kor@leedsbeckett.ac.uk

Abstract

In order to accommodate latency-sensitive IoT and AI workloads, serverless computing is becoming more popular in edge environment. However, the default Kubernetes scheduler ignores the energy and performance limitations of edge nodes and is resource-agnostic. Prior approaches usually only optimized for latency or energy, ignoring the combined effects of cold-start dynamics, inter-node communication, and inter-service dependencies. In this work, we propose a lightweight heuristic scheduling approach that combines inter-service traffic, energy, and latency into a single cost function. This approach, implemented as a custom Kubernetes Scheduling Framework plugin, has low overhead and is used in conjunction with a descheduler that consolidates workloads by draining underutilized nodes. Short-term responsive placements and long-term energy efficiency are made possible by this combination. We test the system on a Raspberry Pi cluster, using Knative workloads that are typical of IoT analytics workflows. The average latency decreased by 29%, failure rates decreased by 74%, and energy consumption per request reduced by 32%, all of which are consistent improvements over the default scheduler. These results show that multi-objective, metrics-aware placement can significantly improve serverless edge platforms' quality of service objectives and energy efficiency, specifically when combined with descheduling for consolidation.

CCS Concepts

• **Computer systems organization** → **Cloud computing**; **Distributed architectures**; • **Software and its engineering** → *Scheduling*; *Virtual machines.*

Keywords

Serverless computing; Edge computing; Kubernetes; Scheduling; Energy efficiency; Latency optimization

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.
UCC '25, Nantes, France
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2285-1/25/12
https://doi.org/10.1145/3773274.3774276

ACM Reference Format:
Shrijan Subedi, Satish Kumar, Nawar Jawad, and Ah Lian Kor. 2025. Energy-Aware Latency Optimization for Scheduling Serverless Workload in Edge Computing Environment. In *2025 IEEE/ACM 18th International Conference on Utility and Cloud Computing (UCC '25), December 01–04, 2025, Nantes, France.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3773274.3774276

1 Introduction

Cloud computing has become the dominant paradigm for delivering scalable and cost-efficient services, providing on-demand access to shared pools of computing resources over the Internet [25]. While this model has proven effective for many applications, cloud-only deployments struggle to meet the requirements of latency-sensitive workloads such as autonomous vehicles, cloud gaming, and large-scale IoT. The long distance to centralized data centers and the massive volumes of data generated at the edge introduce communication delays and bandwidth bottlenecks [9, 35].

Edge computing has emerged as a complementary paradigm that brings computation closer to data sources. By deploying compute resources within access networks, edge computing reduces latency and improves responsiveness for real-time applications. This evolution has been accelerated by advances in wireless access technologies such as 5G, which enable gigabit speeds and millisecond-level latencies [23]. However, edge platforms are fundamentally resource-constrained in terms of CPU, memory, and energy. Sustaining continuous, latency-critical applications require careful trade-offs between responsiveness and efficiency [16].

In parallel, serverless computing, also known as Function-as-a-Service (FaaS), has gained popularity as a model that abstracts deployment and scaling. Functions are invoked on demand, scale automatically, and can even scale to zero during idle periods. While serverless platforms are mature in the cloud, their deployment at the edge is still emerging. The model is well-suited to IoT and interactive workloads, but faces new challenges in this context, including cold-start delays, inter-service communication overheads, and the energy constraints of small-scale devices [34].

Kubernetes, together with Knative [5] and Istio [4], has become the de facto orchestration stack for deploying containerized and serverless workloads across cloud–edge environments. Yet its default scheduler remains largely resource-agnostic, optimizing only for feasibility rather than end-to-end performance or energy efficiency. For serverless pipelines, where functions are short-lived

and highly interactive, overlooking either latency or energy can lead to significant inefficiencies.

This work addresses that gap by developing a heuristic scheduling approach for Kubernetes-based serverless edge clusters. Our approach integrates latency, energy, and inter-service traffic into a unified cost function designed for online decision-making. Implemented as a Kubernetes Scheduling Framework plugin, the customized scheduler operates with minimal overhead alongside a descheduler to further improve energy efficiency through consolidation. We evaluate it on a Raspberry Pi edge cluster running Knative workloads and show that it reduces both latency and energy consumption compared to the default Kubernetes scheduler.

The contributions of this paper are as follows:

- We formulate service placement in serverless edge clusters as a multi-objective decision problem that jointly accounts for latency, energy, and inter-nodes communication.
- We design and implement a lightweight heuristic scheduler integrated into the Kubernetes Scheduling Framework, supported by a metrics-driven decision service.
- We evaluate the scheduler on a physical Raspberry Pi testbed, demonstrating improvements in average latencies as well as energy efficiency over the default scheduler.

The remainder of this paper is structured as follows. Section 2 reviews some background related to energy and latency-aware service placement. Section 3 presents our system design, algorithms and and cost function. Section 4 outlines the evaluation setup and reports the experimental findings. Section 5 reviews some previous work done in the area and Section 6 concludes with directions for future research.

2 Preliminaries

2.1 Service Placement in Edge Computing

Service placement refers to mapping application services onto available infrastructure nodes such that requirements are satisfied within given constraints. Unlike centralized cloud environments, edge systems are geographically distributed, resource-constrained, and heterogeneous. This makes placement a multi-objective problem where goals may include minimizing latency, reducing energy consumption, balancing utilization, or maintaining Quality of Service (QoS) and Quality of Experience (QoE) for end users [32].

The decision of where to deploy a service depends on both the *infrastructure model* (nodes with limited CPU, memory, storage, and networking characteristics) and the *application model* (monolithic services, microservices, or sets of interdependent functions). Serverless platforms, in particular, amplify this challenge due to autoscaling dynamics, cold starts, and dependency chains between functions.

2.1.1 Placement Methods. Placement strategies in distributed and edge systems have been widely studied, and they can be distinguished along several dimensions. Centralized methods leverage a global controller with full system visibility to compute placements, often achieving near-optimal solutions, but they may suffer from scalability bottlenecks and single points of failure. In contrast, distributed approaches delegate decision-making to individual nodes or local agents, thereby enhancing scalability and responsiveness to local conditions at the expense of global optimality. Another key distinction lies between offline and online methods: while offline approaches assume complete knowledge of workloads and system states to pre-compute efficient placements, online approaches dynamically adapt to workload fluctuations and resource availability, a necessity for highly variable edge and serverless environments. Finally, placement can be static, assuming relatively stable workloads, or dynamic, where strategies must accommodate workload bursts, user mobility, or even node failures through runtime migration or rescheduling [20, 32].

In this work, we adopt an online and dynamic placement framework implemented in a centralized controller. This choice reflects the highly variable nature of serverless workloads in edge platforms, where responsiveness to changing latency and energy conditions is critical, but centralized visibility ensures that scheduling decisions can consider global system objectives such as energy reduction and service-level latency guarantees.

2.1.2 Constraints and Challenges. Edge computing environments are subject to several well-known constraints, including limited resources, variable network conditions, application latency requirements, strict energy budgets, and the mobility and heterogeneity of devices [9, 16, 27]. While these factors collectively complicate service placement, this work focuses on the three most critical dimensions for serverless workloads in edge clusters: *resource*, *network*, and *energy* constraints. Resource feasibility is ensured by filtering nodes that cannot satisfy CPU or memory demands. Network constraints are captured through dependency-aware latency modeling, where services that communicate frequently incur penalties if placed on separate nodes, thereby motivating co-location. Energy constraints are integrated via a power model that distinguishes incremental CPU load from idle activation costs, further complemented by a *Descheduler* that consolidates underutilized nodes to reduce idle energy draw. Together, these constraints form the foundation of the proposed scheduling framework, which seeks to balance responsiveness with sustainability in resource-constrained edge platforms.

2.2 Energy and Latency-Aware Scheduling

Scheduling in serverless edge systems often involves a tradeoff between responsiveness and power efficiency. Latency-aware approaches emphasize minimizing response times by monitoring end-to-end delays, integrating resource availability into delay estimates, or classifying workloads for better placement [10, 11, 22]. Energy-aware approaches embed power metrics into placement decisions, for example by optimizing function chains under deadline constraints, adapting workload assignment to nodes' battery state-of-charge, or consolidating workloads to reduce idle consumption [7, 12, 15, 28, 30].

While both methods demonstrate measurable benefits, they often optimize a single objective in isolation. Aggressively minimizing latency can inflate energy consumption due to replication or overprovisioning, while aggressive consolidation to save power can worsen response times. Recent studies propose balancing these competing goals using heuristics or multi-criteria scoring functions

[26, 29, 31]. This trade-off highlights the need for lightweight scheduling plugins that simultaneously consider latency and energy in resource-constrained edge environments.

2.3 Kubernetes Scheduling

Kubernetes is the dominant orchestration system for containerized workloads, automating deployment, scaling, and management across distributed clusters [1]. Its architecture follows a control plane–worker model: the control plane maintains the global state (API server, scheduler, controller manager, and `etcd`), while worker nodes host Pods through local agents (kubelet, container runtime, and kube-proxy).

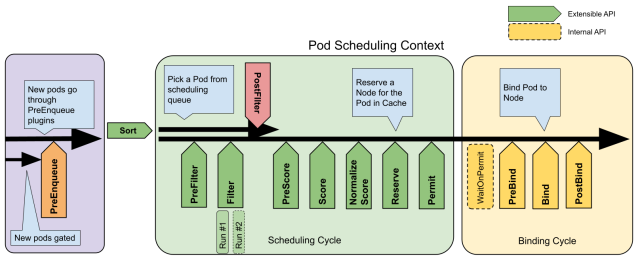

Figure 1: The Kubernetes Scheduling Framework with extension points for custom plugins (adapted from Kubernetes documentation).

The **kube-scheduler** is responsible for assigning Pods to nodes. Its workflow is structured as a pipeline with three main stages:

- *Filtering*: removes nodes that cannot satisfy hard constraints such as CPU/memory requests, taints/tolerations, or affinity rules.
- *Scoring*: evaluates feasible nodes using built-in heuristics (least requested, balanced allocation, affinity-based) and produces a ranking.
- *Binding*: assigns the Pod to the highest-ranked node by updating its `spec.nodeName`.

This process ensures feasibility and load balance, but it does not consider application-level metrics such as latency, energy efficiency, or inter-service communication costs—factors that are critical in edge environments.

To enable customization, Kubernetes exposes the **Scheduling Framework** [17], a plugin architecture with extension points at every stage of the pipeline (e.g., PreFilter, Filter, PreScore, Score, Reserve, Bind). Through these hooks, developers can implement domain-specific policies: for example, scoring nodes by network proximity, prioritizing energy-efficient nodes, or validating application-specific constraints before binding. Because plugins integrate directly with the scheduler's pipeline, they avoid the performance and maintenance issues of older "scheduler extender" APIs.

Recent research prototypes have leveraged this framework to embed reinforcement learning [12], interference-aware scheduling [15], or latency-driven heuristics [10, 21, 22]. These works demonstrate the flexibility of the framework, but most rely on heavy optimization or workload-specific tuning. For serverless edge workloads, where Pod creation is frequent and decisions must be made in milliseconds, lightweight plugins that act on live metrics offer a more practical path.

2.4 Research Motivation

Consider a representative IoT analytics service deployed on an edge cluster: sensor data is ingested and processed at high rate, auxiliary audio streams and images are passed to services that classify events in real time, and a storage holds information and data for other dependent services. Such services are increasingly deployed on constrained edge nodes for smart surveillance, environmental monitoring, or industrial automation. Their correct operation hinges on three interdependent requirements:

- **Low latency.** Sensor and inference services must respond within a few hundred milliseconds to enable timely actuation. If dependent services are placed on different nodes, network delays quickly inflate end-to-end latency. Cold starts further exacerbate this issue by introducing multi-second startup delays.
- **Energy efficiency.** Edge clusters often run on limited power budgets (battery-backed gateways, renewable-powered micro data centers). Keeping underutilized nodes active wastes idle energy, reducing operational lifetime and sustainability.
- **Communication locality.** Many microservices exchange frequent messages. Placing them apart multiplies network traffic, wasting bandwidth and energy while increasing variability in response times.

The default Kubernetes scheduler is ill-suited for such scenarios. Its placement policy considers only resource feasibility and fairness, without accounting for latency, inter-service dependencies, or node energy states. As a result, pods are often scattered across nodes, leading to (i) inflated latencies, (ii) idle nodes that remain powered on unnecessarily, and (iii) high volumes of inter-node traffic.

Existing research partially addresses these concerns: some schedulers focus on energy minimization, others on latency reduction, or on mitigating interference. However, few attempt to integrate all three aspects into a single scheduling policy. Moreover, approaches based on reinforcement learning or integer programming are too computationally heavy for online scheduling in resource-constrained clusters.

This work is motivated by the need for a *lightweight, metrics-driven scheduler* that can operate within the Kubernetes ecosystem and address the challenges of latency, energy, and inter-service communication. By combining real-time observability with a heuristic cost function, the proposed scheduler aims to deliver practical improvements in Quality of Service and energy efficiency for serverless edge workloads.

3 System Design and Problem Modeling

3.1 System Architecture

The proposed system architecture is designed as an extension of the Kubernetes Scheduling Framework, aimed at making placement decisions both responsive and energy-aware. At a high level, the

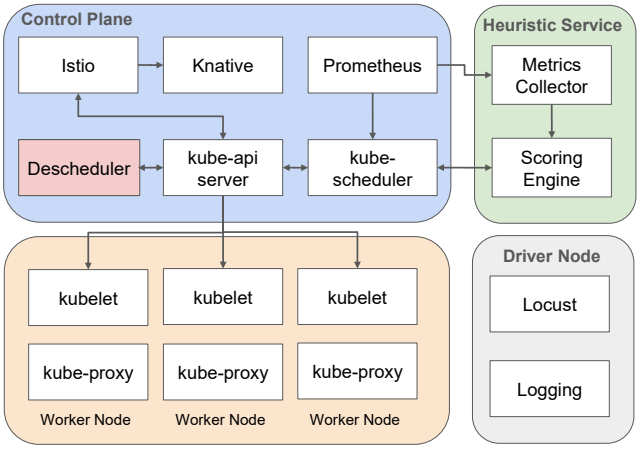

Figure 2: System architecture showing the interaction between the Metrics Collector, Scoring Engine, Descheduler, and Observability Stack.

design integrates four cooperating components: the *Metrics Collector*, which gathers live performance and communication data from the cluster; the *Scoring Engine*, which applies the multi-objective cost function to guide scheduling; the *Descheduler*, which consolidates underutilized nodes to improve energy efficiency; and the *Observability Stack*, which provides the runtime data necessary to maintain accuracy and adaptability. Together, these modules form a continuous feedback loop where real-time metrics drive immediate scheduling choices while background consolidation policies enforce long-term sustainability. The system design emphasizes low overhead, modularity, and practicality for resource-constrained edge environments.

3.1.1 Heuristic Service. The proposed scheduler is supported by two cooperating modules: the *Metrics Collector*, which gathers and normalizes runtime data, and the *Scoring Engine*, which evaluates candidate nodes using the multi-objective cost function defined in Section 3.2. Together, these modules form a continuous feedback loop between the cluster state and scheduling decisions.

Metrics Collector. This module obtains metrics from the observability stack discussed in 3.1.3. The collector aggregates per-container CPU utilization, host-level load, and inter-service communication metrics (request rates, latencies, and traffic volumes). These raw values are normalized to ensure that each scheduling cycle uses consistent, up-to-date inputs. The collector also maintains a *placement map* indicating which pods are currently running on which nodes, allowing co-location ratios for dependent services to be computed on the fly.

Scoring Engine. When a new pod arrives, the scheduling plugin invokes the Scoring Engine via a lightweight REST API. For each candidate node, the engine gathers the following values from the *Metrics Collector*:

(1) *Effective latency* $L_n(s)$, combining base node delay with dependency penalties based on observed request rates and co-location ratios;
(2) *Incremental power cost* $P_n(s)$, estimated from the pod's CPU demand and whether the node is active or idle;
(3) *Normalized scores* $\hat{L}_n(s), \hat{P}_n(s)$ using min–max scaling across all candidates.

These values are combined into the cost function defined in Equation 4. The node with minimal cost is returned to the scheduler, which binds the pod accordingly. This architecture ensures that scheduling decisions remain *metrics-driven* and responsive to the cluster state, while keeping overhead low enough for online decision-making.

3.1.2 Descheduling Policy. While the heuristic cost function (Eq. 4) guides initial placements, cluster utilization may drift over time due to autoscaling and fluctuating workloads. This can leave some nodes underutilized, consuming idle power unnecessarily. To address this, we integrate the Kubernetes Descheduler Project [18] as a complementary module.

The Descheduler runs periodically as a background job, scanning the cluster state and applying a utilization threshold policy. If the CPU utilization of a node falls below 20%, all non-critical pods on that node are evicted and returned to the scheduling queue. These pods are then re-evaluated by the heuristic scheduler, which attempts to consolidate them onto other active nodes. Once a node becomes empty, it can be considered for hibernation, reducing idle energy consumption.

```
deschedulerPolicy:
  profiles:
    - name: consolidate
      pluginConfig:
      - name: "HighNodeUtilization"
        args:
          thresholds:
            cpu: 20
```

This policy-driven mechanism serves as a feedback loop: the scheduler optimizes immediate placement decisions, while the descheduler enforces long-term consolidation. Together, they balance short-term responsiveness with sustained energy efficiency.

3.1.3 Observability Stack. The observability layer provides the runtime metrics needed to evaluate placement decisions and enforce consolidation [21]. Instead of relying on static resource requests, our scheduler queries live metrics at scheduling time, ensuring that decisions reflect the current cluster state.

Prometheus. Prometheus [3] serves as the time-series database and query engine. The Heuristic service queries Prometheus via its HTTP API, ensuring a consistent source of truth for admission-time decisions.

Exporters. Three exporters feed Prometheus with complementary views of the cluster state:

- *cAdvisor* [13], deployed on each node, exposes per-container CPU and memory utilization, forming the basis of the power model.
- *Node Exporter* [2] provides host-level statistics such as aggregate CPU load, memory availability, and thermal readings, which are used to identify underutilized nodes for consolidation.

- *Istio telemetry* captures service-to-service metrics including request rates, latencies, and traffic volumes, enabling dependency-aware latency modeling and co-location analysis.

Metrics flow. When the scheduler receives a pending pod, the Metrics Collector module issues PromQL queries to retrieve per-node utilization, inter-service communication rates, and recent latency measurements. These values are passed to the Scoring Engine, which normalizes them and evaluates the multi-objective cost function. By pulling metrics only at scheduling time, monitoring overhead is kept low enough for Raspberry Pi hardware.

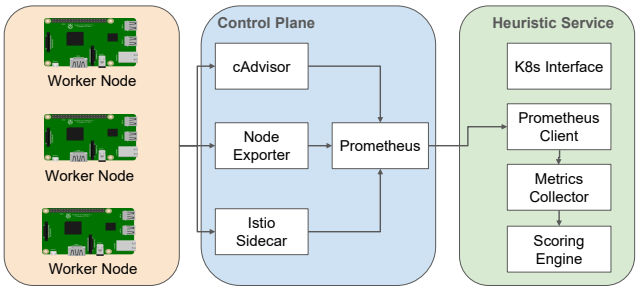

Figure 3: Flow of metrics from various sources

Application-level monitoring. For evaluation purposes, Locust [6] complements the system-level observability stack. It generates workloads and exports request statistics such as average latency, tail latency, and failure rates. These metrics are not used in scheduling decisions but are correlated with Prometheus data to analyze scheduler performance.

3.1.4 Kubernetes and Scheduling Framework. The cluster runs Kubernetes v1.32 with Knative Serving for serverless deployment and Istio as the ingress and service mesh. Knative services allowed us to capture cold-start dynamics through the `minScale` parameter, while Istio sidecars provided fine-grained request telemetry. The proposed scheduler was implemented as a Kubernetes Scheduling Framework plugin, supported by our external *Heuristic Scheduler Service* Section 3.2. Scheduling decisions complete in $O(|N|)$ time and add negligible overhead to the pod admission path.

3.2 Problem Modeling

We cast service placement as a multi-objective decision problem that balances latency and energy. Unlike exact optimization, which is computationally expensive for online scheduling, we design a lightweight heuristic cost function that evaluates candidate nodes in real time. This section elaborates on the latency model, the power model, and the combined cost function used to guide scheduling decisions.

Latency Model. For a candidate node n, the effective latency of placing service s is:

$$L_n(s) = L_n^{\text{base}} + \sum_{d \in D_s} \lambda_{sd} \cdot (1 - \rho_{sd}(n)) \cdot \Delta L_{sd}, \tag{1}$$

where L_n^{base} is the node's base latency, D_s is the set of dependencies of s, λ_{sd} is the observed request rate between s and d, $\rho_{sd}(n)$ is the fraction of colocated pods, and ΔL_{sd} is the penalty when calls cross nodes.

This formulation captures two important aspects. First, every node has an inherent baseline latency, which may reflect processing capability, queueing delay, or network proximity to end users. Second, serverless functions often operate as part of a pipeline, where the placement of dependent services directly impacts end-to-end latency. If two services communicate frequently, placing them on different nodes adds a network penalty proportional to their request rate. Thus, Eq. 1 naturally promotes the colocation of tightly coupled services, while allowing loosely coupled ones to be distributed without incurring significant cost.

Power Model. The incremental power draw of assigning s to n is estimated as:

$$P_n(s) = \begin{cases} \kappa \cdot \dfrac{C_s}{N_{\text{cores}}}, & \text{if } n \text{ already active,} \\ \kappa \cdot \dfrac{C_s}{N_{\text{cores}}} + P_{\text{idle}}, & \text{if } n \text{ is idle,} \end{cases} \tag{2}$$

with C_s the CPU demand of s, N_{cores} the cores per node, $\kappa = 4.5344$ the regression slope, and $P_{\text{idle}} = 2.2857$ W the idle offset [14].

This model distinguishes between two scenarios. If the node is already active, the additional energy cost comes only from the incremental CPU load of the new pod. If the node is idle, however, scheduling a single pod forces the system to pay an additional "activation cost," represented by P_{idle}. This encourages the scheduler to consolidate workloads on active nodes where possible, and only activate new nodes when necessary.

Heuristic Cost. Latency and power are measured in different units (milliseconds vs. watts), so we normalize them into the range $[0, 1]$ before combining [19]:

$$\hat{L}_n(s) = \frac{L_n(s) - \min(L)}{\max(L) - \min(L)}, \qquad \hat{P}_n(s) = \frac{P_n(s) - \min(P)}{\max(P) - \min(P)}. \tag{3}$$

The final cost for placing service s on node n is a convex combination of normalized latency and power, following the common weighted-sum approach used in multi-objective scheduling frameworks [26, 31]:

$$C_n(s) = \alpha \cdot \hat{L}_n(s) + \beta \cdot \hat{P}_n(s), \qquad \alpha + \beta = 1, \tag{4}$$

where α and β represent the relative importance of latency and energy, respectively. In our experiments we set $\alpha = 0.7$ and $\beta = 0.3$ to reflect latency-sensitive workloads. Finally, the scheduler selects the node with the lowest cost:

$$n^* = \arg\min_{n \in N} C_n(s). \tag{5}$$

By explicitly modeling dependencies, activation costs, and trade-offs between metrics, the cost function provides a principled yet efficient basis for scheduling decisions in serverless edge clusters.

3.3 Algorithm

The placement algorithm proceeds in two stages. First, runtime metrics are collected from Prometheus, including per-node latency and pod CPU utilization. Then, each candidate node is scored by normalizing latency and power and computing a weighted sum. The node with the lowest score is selected. The procedure is outlined in Algorithm 1.

Algorithm 1 Heuristic Placement Algorithm

Input: Pending service pod s, Candidate nodes N
Output: Selected node n^*

```
for each candidate node n ∈ N do
    if n does not have sufficient resources then
        skip to next node
    end if
    Query Prometheus for:
        L_n: Effective latency of placing s on node n
        P_n: Incremental power cost of placing s on node n
    Normalize L_n and P_n across all candidates
    Compute score:
        C_n = α · L̂_n + β · P̂_n
end for
Select node n* with minimal C_n
return Node n* to scheduler
```

To complement this process, a Descheduler runs in parallel to identify nodes with low utilization. By evicting pods from such nodes and re-submitting them for placement, the system is able to consolidate workloads and shut down idle nodes. The logic is given in Algorithm 2.

Algorithm 2 Descheduling Algorithm

Input: Set of nodes N with running pods
Output: Consolidated placement

```
for each node n ∈ N do
    Query Prometheus for:
        U_n: Resource utilization of node n
    if U_n < U_threshold then
        for each pod p on node n do
            if p is not critical and can be rescheduled then
                Evict pod p
            end if
        end for
        if n has no more pods then
            Mark n for hibernation
        end if
    end if
end for
```

The combination of scheduler and descheduler thus creates a feedback loop: the scheduler prioritizes latency in immediate decisions, while the descheduler gradually enforces consolidation to reduce energy consumption.

4 Experimental Evaluation

4.1 Experimental Setup

Experiments were conducted on a physical Raspberry Pi cluster consisting of one control-plane node and three worker nodes. Each device was a Raspberry Pi 4B with a quad-core Cortex-A72 CPU, 8 GB RAM, and 128 GB storage, running Ubuntu 22.04 LTS. The nodes were interconnected over a dedicated Wi-Fi network, representing a resource-constrained yet realistic edge setting.

The workload emulates an IoT analytics service of five microservices: *Inference* (YOLOv4-Tiny [8]), *Storage* (model repository), *Sensor Process*, *Sensor Flood* (high-rate traffic), and *Audio Process*. Dependencies between services (e.g., Inference–Storage, Sensor Flood–Sensor Process) introduce inter-node traffic, allowing evaluation of colocation effects.

Traffic was generated with Locust [6], with separate user classes mapped to service endpoints. The tool controlled concurrency and ramp-up, and exported per-request statistics (latency, throughput, failures) for post-processing.

4.2 Results and Discussion

4.2.1 Characterizing Serverless Edge Behavior. We first examined baseline serverless behavior on the Raspberry Pi cluster. Table 1 reports idle pod consumption. Although each service individually consumed under 0.05 W, five always-on pods together drew 0.20 W and over 550 MiB of memory. In absolute terms this footprint may seem small, but in a three-node cluster such background draw is notable: leaving idle services resident wastes both energy and memory capacity that could otherwise serve active workloads. This highlights the importance of scale-to-zero, especially when many microservices are deployed in parallel.

Table 1: Resource usage of pods in idle state

Service	CPU (s)	Power (W)	Memory (MiB)
Audio Process	0.0092	0.0417	86.7
Inference	0.0096	0.0435	93.4
Sensor Flood	0.0094	0.0425	112.9
Sensor Process	0.0092	0.0417	80.6
Storage	0.0074	0.0336	180.4
Total	0.0448	0.2030	554.1

Cold starts introduced noticeable penalties. With `minScale=0`, average inference latency exceeded 2.4 s, and maximums stretched past 12 s. Sensor Flood also failed in one run because its dependent service was not yet active, while Audio Process showed long delays but no outright failures. Table 2 summarizes these results, showing both high variability and the potential for cascading errors when dependencies are cold.

Table 2: Cold-start response metrics (`minScale=0`)

Service	Failures	Avgerage (ms)	Maximum (ms)
Inference	0	2416.6	12589.9
Sensor Flood	1	1334.2	17342.3
Audio Process	0	1371.9	11036.6

Pre-warming pods (`minScale=1`) sharply reduced these penalties. As shown in Table 3, average inference latency fell to 1.7 s, maximum latency dropped by an order of magnitude, and failures disappeared entirely. Sensor Flood in particular benefited, with mean response times falling from 1.3 s to 0.3 s. The trade-off is clear: keeping one warm replica consumes more idle energy, but it avoids multi-second stalls and dependency-related failures, making workloads more reliable and predictable.

Table 3: Warm-start response metrics (`minScale=1`)

Service	Failures	Average (ms)	Maximum (ms)
Inference	0	1686.6	2057.4
Sensor Flood	0	309.5	1115.4
Audio Process	0	823.1	1493.4

Placement effects were just as important as cold starts. Co-locating services that exchange large volumes of data avoided cross-node transfers and cut response times substantially. Inference-to-Modeldepot latency dropped from 10.1 s to 1.2 s when co-located, while Sensor Flood-to-Sensor Process latency fell by nearly half. More strikingly, 24 MB of network traffic per request was eliminated in the Inference case, and over 1 MB in the Sensor Flood case, as shown in Tables 4– 5. In an edge cluster with constrained bandwidth, these savings directly translate into reduced network contention and energy overhead.

Table 4: Inter-service communication with co-location

Source	Destination	Latency (s)	Traffic (Bytes)
Inference	Modeldepot	1.22	0
Sensor Flood	Sensor Proc	1.89	0

Table 5: Inter-service communication without co-location

Source	Destination	Latency (s)	Traffic (Bytes)
Inference	Modeldepot	10.09	24,255,131
Sensor Flood	Sensor Proc	2.31	1,281,787

Taken together, these results confirm that idle pods, cold starts, and poor placement each introduce non-trivial overheads in serverless edge systems. Idle pods inflate the background energy and memory footprint, cold starts degrade responsiveness and can trigger failures, and cross-node placement multiplies latency and traffic. These characterizations motivate the need for schedulers that are explicitly aware of energy costs, cold-start penalties, and service dependencies—concerns that we address in the next section with our heuristic scheduler.

4.2.2 Scheduler Evaluation. We then evaluated our scheduler against the default Kubernetes scheduler across ten trials. Energy efficiency results are shown in Table 6. The custom scheduler averaged 0.00122 Wh/request compared to 0.00179 Wh/request under default—a 32% reduction. Average worker power draw dropped from 2.14 Wh to 1.95 Wh per trial.

Looking more closely at the energy results, the difference is consistent across all runs: even the least efficient trial with our custom scheduler consumed less energy per request than the best trial under the default scheduler. This indicates that the savings are not an artifact of a few lucky placements but a systematic effect of consolidation and co-location. The variance in energy also narrowed, with the interquartile range for custom runs being smaller than for the default, showing that the heuristic improves predictability in addition to lowering the mean.

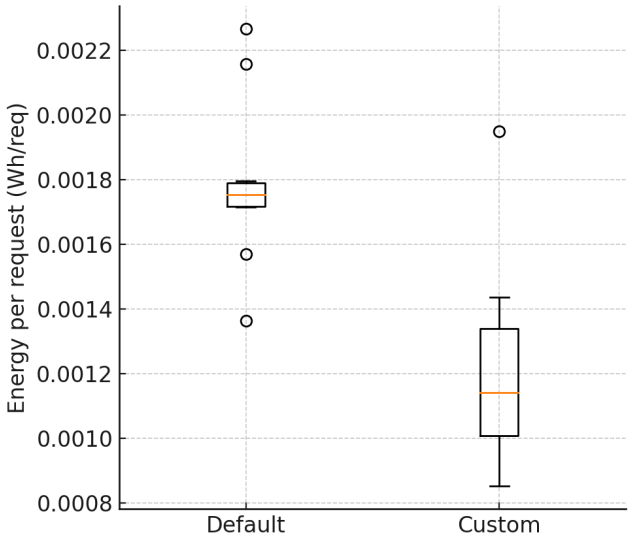

Figure 4: Energy consumption per request across ten experimental trials, comparing the default Kubernetes scheduler with the proposed heuristic scheduler.

Latency likewise improved. Figure 5 shows the distribution of average request latencies: the heuristic scheduler reduced the mean from 5476 ms to 3890 ms (≈1.6 s faster). Failures fell from 10.1% under default to 2.6% with the custom scheduler.

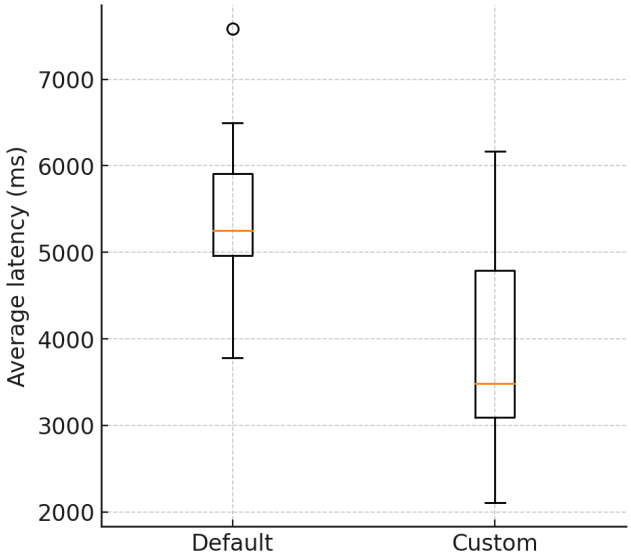

Figure 5: Distribution of average request latencies across ten experimental trials, comparing the default Kubernetes scheduler with the proposed heuristic scheduler.

The latency improvements were most visible for services with strong inter-service dependencies. For instance, the Sensor Flood service, which consistently suffered high delays and failures under

the default scheduler, saw its average latency drop by more than 3 s and its failures reduced by more than half when scheduled with our heuristic. The Inference service also benefited, with a reduction of roughly 3.4 s in mean response time. Audio Process, while less sensitive to placement, still gained almost 1.3 s in responsiveness. This shows that the scheduler not only reduces overall averages but also distributes improvements across different service types.

Table 6 summarizes these aggregate gains. Unlike reinforcement learning schedulers such as Smart-Kube [12], our heuristic introduces minimal overhead and no training cost, making it practical for low-power edge clusters.

Table 6: Comparison of default vs. custom scheduler (averages across 10 trials)

Metric	Default	Custom	Improvement
Energy/request (Wh)	0.00179	0.00122	32%
Avg latency (ms)	5476	3890	29%
Failure rate (%)	10.1	2.6	74%

Beyond averages, the tail behavior also improved. Under the default scheduler, maximum latencies frequently exceeded 40 s, with one run peaking above 58 s. With the custom scheduler, these extreme outliers were less frequent, and while some trials still saw high values (>60 s), the majority of requests were served much faster. Thus, the custom scheduler both reduced mean latencies and tightened the spread of response times, improving the overall quality of service experienced by users.

In summary, across energy, latency, and reliability metrics, the custom scheduler consistently outperformed the default. The reductions are large enough to matter in practice: saving one third of energy per request directly extends device lifetime, while cutting latency by over a second per request can be the difference between a tolerable and a frustrating user experience.

4.3 Discussion

The results show that even a lightweight heuristic can jointly improve energy efficiency, responsiveness, and reliability in a Kubernetes-based serverless edge platform. Across ten trials, the custom scheduler consistently reduced energy-per-request by about one third and lowered average latency by nearly 1.6 s compared to the default scheduler. These improvements are not only statistically significant but also practically relevant in edge deployments where both battery life and user-perceived QoS are critical.

Energy savings arose primarily from consolidation and the ability to deschedule idle nodes. By steering workloads toward fewer active workers when demand was low, the scheduler reduced the background energy draw that otherwise accumulated in underutilized nodes. The reduction in variance across trials also suggests that the heuristic makes system behavior more predictable, which is particularly valuable in constrained clusters.

Latency improvements were driven largely by dependency-aware co-location. Services such as *Inference* and *Sensor Flood* benefited the most: colocating them with their storage or processing dependencies cut multi-second penalties and sharply reduced failures. Even *Audio Process*, which was less sensitive to placement, saw consistent reductions. These gains show that modest awareness of service interactions can translate into broad improvements across diverse workloads.

Reliability also improved. Failure rates fell from over 10% to under 3%, with the steepest reduction in the Sensor Flood service, which was previously prone to dependency timeouts. While extreme tail latencies were not eliminated—some trials still saw spikes above 60 s—the frequency of such outliers was lower and the distribution of latencies was tighter overall. In practice, this means that most users would experience smoother and more predictable service, even if rare stragglers remain.

Taken together, these findings provide evidence that multi-objective scheduling, even in simple heuristic form, is effective in serverless edge environments. The approach balances energy and performance without the overhead of complex learning-based methods, making it well-suited for clusters built from resource-constrained devices such as Raspberry Pis. Although further work is needed to address long-tail latencies and to validate at larger scales, the study demonstrates that meaningful gains can be achieved by incorporating just a small amount of energy- and communication-awareness into scheduling decisions.

4.4 Threats to Validity

While the evaluation provides useful insights, several factors may limit the generalization of the results:

(1) **Scale.** Results are limited to a small three-node Raspberry Pi cluster; larger and more diverse setups remain untested.
(2) **Energy model.** The CPU-focused model ignores other components (Network, storage, or memory), so absolute energy values may be biased, though comparisons remain valid.
(3) **Workload mix.** Evaluation focused on IoT-style analytics; validation on more diverse workloads is needed.
(4) **Tail latency.** High tail latencies were observed, and further work is needed to reduce these extremes.

5 Related Works

In this section, we discuss the most relevant studies in greater detail and position our approach with respect to them.

5.1 Latency-Centric Approaches

Several works emphasize responsiveness as the primary metric for scheduling. A latency-aware Kubernetes scheduler was proposed that measures end-to-end application delays to guide placement, achieving measurable reductions in response times but focusing exclusively on latency [10]. Another design, Delay-Aware Container Scheduling (DACS), incorporates both delay and resource availability to improve performance in heterogeneous clusters [22]. Dynamic placement based on workload profiling and classification has also been explored for heterogeneous Kubernetes clusters [11]. These studies demonstrate that delay-driven policies improve responsiveness but do not address energy budgets or serverless-specific dynamics such as cold-start penalties.

5.2 Network-Aware Approaches

Interactive microservices often incur high overheads when placed across nodes. One study introduced a network-aware scheduler and

descheduler pair that used communication metrics to co-locate dependent services and improve responsiveness [24]. Another work, NetMARKS, integrated service-mesh telemetry to prioritize placement on low-latency links and reduce bandwidth usage [33]. While effective for minimizing communication costs, these approaches optimize primarily for network state without considering power draw or long-term energy efficiency.

5.3 Energy-Centric Approaches

A complementary line of work has placed energy efficiency at the forefront. EneX formulated the placement of function chains with deadlines as an optimization problem, showing reductions in execution energy through an online heuristic [30]. FaasHouse targeted renewable-powered edge clusters by balancing workload assignment against node state-of-charge to improve sustainability [7]. Integer programming approaches have been applied to minimize both energy use and interference in industrial IoT clusters [15], while swarm-inspired meta-heuristics have been used to reduce energy consumption in Kubernetes clusters with SLA constraints [29]. These works demonstrate energy savings but often neglect latency-critical requirements or incur optimization overheads that limit real-time deployment.

5.4 Multi-Objective and Learning-Based Approaches

To address trade-offs between latency, energy, and utilization, several works have explored multi-objective frameworks. One study introduced HEATS, which combines workload prediction with user-defined trade-offs between energy and performance [31]. Multi-criteria container schedulers have been proposed that explicitly integrate power as one of several objectives [26]. Reinforcement learning has also been applied, as in Smart-Kube, which jointly optimizes utilization and energy fairness but requires costly training and model maintenance [12]. Although these methods achieve broader optimization, their reliance on computationally heavy solvers or training complexity makes them less suitable for resource-constrained serverless edge platforms where scheduling decisions must be taken online and within milliseconds.

Despite significant progress in extending Kubernetes scheduling for edge computing, prior studies remain limited in scope. Most optimize a single dimension—such as latency, network, or energy—without addressing the trade-offs that arise in multi-objective settings. Only a few support fully online scheduling, and even fewer consider the distinctive demands of serverless workloads, including cold-start delays and scale-to-zero dynamics. Moreover, many approaches rely on simulations rather than real deployments, reducing their practical relevance for heterogeneous, resource-constrained clusters.

To address these gaps, this work proposes a lightweight, online scheduling framework that jointly optimizes latency, energy, and inter-service traffic using live runtime metrics. Implemented as a Kubernetes Scheduling Framework plugin and validated on a physical Raspberry Pi edge cluster, the approach demonstrates consistent improvements in both responsiveness and efficiency under realistic conditions.

6 Conclusion and Future Work

This paper presented a lightweight heuristic scheduler for Kubernetes-based serverless edge environments. The scheduler integrates latency, energy, and inter-service communication into a unified cost function, implemented as a Kubernetes Scheduling Framework plugin. When evaluated on a Raspberry Pi cluster, our scheduler demonstrated consistent improvements over the default scheduler: average latency decreased by nearly 30%, failures dropped by more than two-thirds, and energy per request was reduced by approximately 32%. These results confirm that metrics-aware placement can substantially improve both quality of service and energy efficiency without incurring significant runtime overhead.

Several directions remain for future work. First, while our heuristic uses fixed weights to balance latency and energy, adaptive weighting based on workload type, service-level objectives, or renewable energy availability could provide further flexibility. Second, our evaluation was limited to a small three-node edge cluster; larger clusters with heterogeneous resources and diverse workloads are needed to assess scalability. Third, the current energy model is CPU-centric; extending it to incorporate memory, storage, and network subsystems would improve accuracy. Finally, integrating predictive or learning-based components for proactive scheduling could further reduce tail latency while retaining the low overhead of heuristic methods.

Overall, the findings demonstrate that practical, lightweight schedulers can deliver significant performance and energy benefits for serverless edge computing, pointing toward more sustainable and responsive cloud–edge systems.

Acknowledgments

This research work is supported and implemented under the framework of the Erasmus Mundus Joint Masters Degree (EMJMD) in Green Networking and Cloud Computing (GENIAL) program (610619-EPP-1-2019-1-FR-EPPKA1-JMD-MOB).

References

[1] The Kubernetes Authors 2025. *Kubernetes Documentation.* The Kubernetes Authors. https://kubernetes.io/docs/concepts/overview/#why-you-need-kubernetes-and-what-can-it-do
[2] Prometheus Authors 2023. *Node Exporter - Prometheus Documentation.* Prometheus Authors. https://prometheus.io/docs/guides/node-exporter/
[3] Prometheus Authors 2023. *Prometheus - Overview.* Prometheus Authors. https://prometheus.io/docs/introduction/overview/
[4] Istio Authors 2024. *What is Istio?* Istio Authors. https://istio.io/latest/docs/overview/what-is-istio/
[5] 2025. Knative Concepts. https://knative.dev/docs/concepts/. [Online; accessed 17-September-2025].
[6] 2025. Locust Documentation. https://docs.locust.io/en/stable/. [Online; accessed 17-September-2025].
[7] Mohammad Sadegh Aslanpour, Adel N Toosi, Muhammad Aamir Cheema, and Mohan Baruwal Chhetri. 2024. FaasHouse: sustainable serverless edge computing through energy-aware resource scheduling. *IEEE Transactions on Services Computing* 17, 4 (2024), 1533–1547.
[8] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. 2020. Yolov4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934* (2020).
[9] Keyan Cao, Yefan Liu, Gongjie Meng, and Qimeng Sun. 2020. An overview on edge computing research. *IEEE access* 8 (2020), 85714–85728.
[10] Carlo Centofanti, Walter Tiberti, Andrea Marotta, Fabio Graziosi, and Dajana Cassioli. 2023. Latency-aware kubernetes scheduling for microservices orchestration at the edge. In *2023 IEEE 9th International Conference on Network Softwarization (NetSoft).* IEEE, 426–431.
[11] Ghofrane El Haj Ahmed, Felipe Gil-Castiñeira, and Enrique Costa-Montenegro. 2021. KubCG: A dynamic Kubernetes scheduler for heterogeneous clusters.

Software: Practice and Experience 51, 2 (2021), 213–234.

[12] Saeid Ghafouri, Sina Abdipoor, and Joseph Doyle. 2023. Smart-kube: Energy-aware and fair kubernetes job scheduler using deep reinforcement learning. In *2023 IEEE 8th International Conference on Smart Cloud (SmartCloud)*. IEEE, 154–163.

[13] Google. 2025. cAdvisor: container advisor for resource and performance metrics. https://github.com/google/cadvisor. [Online; accessed 17-September-2025].

[14] Houssam Kanso, Adel Noureddine, and Ernesto Exposito. 2023. Automated power modeling of computing devices: Implementation and use case for Raspberry Pis. *Sustainable Computing: Informatics and Systems* 37 (2023), 100837.

[15] Kuljeet Kaur, Sahil Garg, Georges Kaddoum, Syed Hassan Ahmed, and Mohammed Atiquzzaman. 2019. KEIDS: Kubernetes-based energy and interference driven scheduler for industrial IoT in edge-cloud ecosystem. *IEEE Internet of Things Journal* 7, 5 (2019), 4228–4237.

[16] Linghe Kong, Jinlin Tan, Junqin Huang, Guihai Chen, Shuaitian Wang, Xi Jin, Peng Zeng, Muhammad Khan, and Sajal K Das. 2022. Edge-computing-driven internet of things: A survey. *Comput. Surveys* 55, 8 (2022), 1–41.

[17] Kubernetes. 2025. *Scheduling Framework*. The Kubernetes Authors. https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/ Accessed: 2024-09-11.

[18] Kubernetes SIG-Scheduling. 2024. Kubernetes Descheduler. https://github.com/kubernetes-sigs/descheduler. Accessed: 2025-08-02.

[19] Satish Kumar, Rami Bahsoon, Tao Chen, Ke Li, and Rajkumar Buyya. 2018. Multi-tenant cloud service composition using evolutionary optimization. In *2018 IEEE 24th international conference on parallel and distributed systems (ICPADS)*. IEEE, 972–979.

[20] Satish Kumar, Tao Chen, Rami Bahsoon, and Rajkumar Buyya. 2023. DebtCom: Technical debt-aware service recomposition in SaaS cloud. *IEEE Transactions on Services Computing* 16, 4 (2023), 2545–2558.

[21] Satish Kumar, Nawar Jawad, and Rami Bahsoon. 2025. Locality-Aware QoS Optimization for Microservices Scheduling in Kubernetes Cluster. In *2025 IEEE International Conference on High Performance Computing and Communications (HPCC)*. IEEE, 544–551.

[22] Wei-Kuang Lai, You-Chiun Wang, and Syu-Chen Wei. 2023. Delay-aware container scheduling in kubernetes. *IEEE Internet of Things Journal* 10, 13 (2023), 11813–11824.

[23] Li Lin, Xiaofei Liao, Hai Jin, and Peng Li. 2019. Computation offloading toward edge computing. *Proc. IEEE* 107, 8 (2019), 1584–1607.

[24] Angelo Marchese and Orazio Tomarchio. 2022. Network-aware container placement in cloud-edge kubernetes clusters. In *2022 22nd IEEE international symposium on cluster, cloud and internet computing (CCGrid)*. IEEE, 859–865.

[25] Peter Mell, Tim Grance, et al. 2011. The NIST definition of cloud computing. (2011).

[26] Tarek Menouer. 2021. KCSS: Kubernetes container scheduling strategy. *The Journal of Supercomputing* 77, 5 (2021), 4267–4293.

[27] Clément Mommessin, Renyu Yang, Natalia V Shakhlevich, Xiaoyang Sun, Satish Kumar, Junqing Xiao, and Jie Xu. 2023. Affinity-aware resource provisioning for long-running applications in shared clusters. *J. Parallel and Distrib. Comput.* 177 (2023), 1–16.

[28] Tobias Piontek, Kawsar Haghshenas, and Marco Aiello. 2024. Carbon emission-aware job scheduling for Kubernetes deployments. *The Journal of Supercomputing* 80, 1 (2024), 549–569.

[29] Wei Rao and Hongjian Li. 2025. Energy-aware Scheduling Algorithm for Microservices in Kubernetes Clouds. *Journal of Grid Computing* 23, 1 (2025), 1–22.

[30] Seyed Hamed Rastegar, Hossein Shafiei, and Ahmad Khonsari. 2023. Enex: An energy-aware execution scheduler for serverless computing. *IEEE Transactions on Industrial Informatics* 20, 2 (2023), 2342–2353.

[31] Isabelly Rocha, Christian Göttel, Pascal Felber, Marcelo Pasin, Romain Rouvoy, and Valerio Schiavoni. 2019. Heats: Heterogeneity-and energy-aware task-based scheduling. In *2019 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*. IEEE, 400–405.

[32] Farah Ait Salaht, Frédéric Desprez, and Adrien Lebre. 2020. An overview of service placement problem in fog and edge computing. *ACM Computing Surveys (CSUR)* 53, 3 (2020), 1–35.

[33] Łukasz Wojciechowski, Krzysztof Opasiak, Jakub Latusek, Maciej Wereski, Victor Morales, Taewan Kim, and Moonki Hong. 2021. Netmarks: Network metrics-aware kubernetes scheduler powered by service mesh. In *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*. IEEE, 1–9.

[34] Renchao Xie, Qinqin Tang, Shi Qiao, Han Zhu, F Richard Yu, and Tao Huang. 2021. When serverless computing meets edge computing: Architecture, challenges, and open issues. *IEEE Wireless Communications* 28, 5 (2021), 126–133.

[35] Ashkan Yousefpour, Caleb Fung, Tam Nguyen, Krishna Kadiyala, Fatemeh Jalali, Amirreza Niakanlahiji, Jian Kong, and Jason P Jue. 2019. All one needs to know about fog computing and related edge computing paradigms: A complete survey. *Journal of Systems Architecture* 98 (2019), 289–330.